

وَقُلْنَا يَا مَعْشَرَ الْعُلَمَاءِ

سُبْحَانَكَ لَا عِلْمَ لَنَا إِلَّا مَا عَلَّمْتَنَا إِنَّكَ أَنْتَ الْعَلِيمُ الْحَكِيمُ

Dedications

In the name of Allah, the Most Gracious, the Most Merciful

To our beloved parents,

*whose unwavering support, endless sacrifices, and constant prayers
have been the foundation of our success.*

*To our teachers and mentors,
who guided us with knowledge and wisdom
throughout our academic journey.*

*To the team at Novation City,
who welcomed us with open arms and provided
invaluable learning opportunities.*

*To our families and friends,
whose encouragement and understanding
made this achievement possible.*

With deep gratitude and appreciation.

Acknowledgments

I would like to express my sincere gratitude to all those who contributed to the success of this professional internship experience, including academic supervisors, industry mentors, and institutional staff whose guidance, feedback, and practical support were essential to achieving the project objectives.

First, I extend my deepest appreciation to my academic supervisor, **Mr. Ahmed Bin Ayed**, for their invaluable guidance and continuous support throughout this project. Their expertise was instrumental in ensuring academic rigor and alignment with university requirements.

I am profoundly grateful to the Novation City team who welcomed me into their innovative ecosystem. Special thanks to my professional supervisor, **Mr. Mohamed Chrifa**, for their professional insights and technical guidance in Industry 4.0 implementation and digital transformation consulting.

My sincere appreciation goes to Novation City competitiveness cluster for entrusting me with developing the **IoT-REAP (Internet of Things - Remote Equipment Access Platform)**. Building this secure industrial remote operations platform—integrating Proxmox VE virtualization, Apache Guacamole remote desktop, MQTT-based IoT robot control, and AI-powered demand forecasting—has been an invaluable experience for my professional growth in full-stack development and industrial systems integration.

I extend gratitude to EPI Digital School’s administration and faculty for providing quality education and academic resources essential for understanding enterprise-grade web application development, security architecture, and modern software engineering practices.

Finally, I am grateful to my family and friends for their unwavering support and encouragement throughout this journey.

Contents

Dedications	i
Acknowledgments	ii
List of Figures	v
List of Tables	vi
Chapter 1: Preliminary Study	1
Introduction	2
1 General Project Context	2
1.1 Host Organization	2
1.2 Business Units and Services	3
1.3 Infrastructure and Strategic Focus	3
1.4 Academic Framework	4
1.5 Problem Statement	4
2 Study of the Existing	6
2.1 Analysis of Existing Industrial Remote Access Solutions	6
2.2 Criticism of the Existing	6
3 Proposed Solution	8
3.1 Project Definition	8
3.2 Solution Architecture	8
4 Requirements Specification	9
4.1 Identification of Actors	9
4.2 Functional Requirements	10
4.3 Non-Functional Requirements	13
5 Work Methodology	15
5.1 The Agile Scrum Methodology	15
5.1.1 Definition	15
5.1.2 Characteristics of Agile Scrum	16
5.2 Product Backlog	17
5.3 Agile Scrum Ceremonies and Collaboration	21
5.4 The UML Modeling Language	22
6 Project Planning	23
6.1 Sprint-Based Development Phases	23
Conclusion	26

Webgraphy	27
Abstract	33

List of Figures

1.1	Novation City Official Logo and Branding	2
1.2	Agile Scrum Methodology Workflow	16
1.3	UML modeling language	22
1.4	IoT-REAP Platform Development Timeline and Milestones	25

List of Tables

1.1	Critical Limitations of Current Industrial Remote Access Solutions	5
1.2	Comparison of Existing Industrial Remote Access Solutions	7
1.3	IoT-REAP Platform Product Backlog	17

Chapter 1

Preliminary Study

Introduction

This chapter presents the preliminary study of the IoT-REAP project. It situates the work in the Novation City context, reviews existing industrial remote access solutions, defines the proposed platform and its requirements, and ends with the methodology and planning used to structure the implementation.

1 General Project Context

Novation City serves as the professional context for the IoT-REAP platform development. This section situates the project within the cluster's institutional framework, business units, infrastructure, and academic setting, all of which explain why a secure remote industrial access platform was needed.

1.1 Host Organization

Novation City is a competitiveness cluster established in 2009 in Sousse, Tunisia, as a public-private partnership. The cluster is designed to foster innovation ecosystems and enhance industrial competitiveness through digital transformation and advanced technologies, addressing the growing need for mechatronics expertise and Industry 4.0 adoption in the MENA region [1].



Figure 1.1: Novation City Official Logo and Branding

Mission Statement: Novation City's mission is to foster an innovation ecosystem in the mechatronics sector by bringing together companies, industrial entities, research centers, universities, and engineering schools. The cluster aims to enhance regional and national competitiveness through technology advancement, knowledge transfer, and strategic investment attraction.

Vision: The cluster envisions becoming a leading competitiveness hub for Industry 4.0 adoption and digital transformation in North Africa and the broader MENA region, focusing

on innovation infrastructure, technical talent development, and sustainable academia-industry partnerships.

1.2 Business Units and Services

Novation City operates through four specialized business units, each addressing distinct aspects of the innovation ecosystem:

Acceleration Unit: Supports startup growth through business incubation services, mentorship programs, and access to funding opportunities. Services include workspace provision, networking, and investor connections.

Advisory Unit: Delivers strategic consulting focused on digital transformation and Industry 4.0 implementation. The advisory services encompass digital readiness assessment, technology selection, change management, security compliance audits, and infrastructure planning for remote operations platforms.

Formation Unit (Novation Akademy): Provides training and skills development programs covering mechatronics, automation, digital technologies, project management, and innovation methodologies in collaboration with universities and industry partners.

Innovation Unit: Drives research and development initiatives, facilitating collaborative R&D between academic institutions and industrial partners while exploring emerging technologies for the regional ecosystem.

1.3 Infrastructure and Strategic Focus

Novation City manages three specialized geographic zones:

- **Mechatronic City:** Houses companies focused on mechatronics, robotics, and advanced manufacturing with specialized laboratories and testing facilities.
- **Business City:** Accommodates service-oriented companies and consulting firms with modern office spaces and collaborative environments.
- **Industrial City:** Dedicated to manufacturing operations with infrastructure supporting advanced manufacturing processes and Industry 4.0 technologies.

A central pillar of Novation City's strategy is promoting Industry 4.0 principles and digital transformation. The cluster recognizes that many industrial companies face significant challenges in implementing Industry 4.0 technologies, including limited awareness, uncertainty about ROI, skills gaps, and integration difficulties with legacy systems.

To address these challenges, Novation City extends its capabilities with secure remote access platforms enabling engineers to interact with industrial systems remotely through the IoT-REAP initiative, combining Infrastructure-as-a-Service provisioning with zero-trust security principles.

1.4 Academic Framework

The project fits within the software engineering and Industry 4.0 curriculum, applying web engineering, software architecture, database modeling, remote infrastructure integration, and secure system design to an industrial problem posed by Novation City. The academic evaluation framework requires formal documentation through UML modeling, requirements analysis, implementation reporting, and verification of the delivered platform.

1.5 Problem Statement

The IoT-REAP project addresses critical operational challenges facing industrial companies and training institutions within the Industry 4.0 ecosystem. As manufacturing systems become increasingly digitized and interconnected, engineers and technicians require hands-on access to specialized industrial equipment—PLCs (Programmable Logic Controllers), HMIs (Human-Machine Interfaces), SCADA systems, and industrial robots—for training, development, and maintenance purposes.

Remote Access Limitations: Traditional approaches to industrial system access present significant challenges:

- Physical presence requirements limit training scalability and increase travel costs
- VPN-based solutions are complex to configure, difficult to maintain, and often incompatible with OT (Operational Technology) security requirements
- Existing remote desktop tools lack integration with industrial protocols and equipment
- No unified platform exists for VM provisioning, remote desktop access, and IoT device control

Security Concerns in OT Environments: Industrial networks require strict isolation following the Purdue model, yet traditional IT remote access tools are not designed for OT security requirements:

- **Network Segmentation:** IT remote access solutions often bypass OT network boundaries, creating security vulnerabilities

- **Session Management:** No automatic session expiration or quota management for shared industrial resources
- **Audit Trail:** Insufficient logging for compliance with industrial security standards (IEC 62443) [5]
- **Access Control:** Lack of fine-grained role-based access to different industrial zones

Resource Sharing Challenges: Educational institutions and industrial companies face difficulties in sharing expensive equipment:

- **Equipment Scarcity:** Industrial robots and specialized VMs are costly and limited in quantity
- **Scheduling Conflicts:** No automated reservation system prevents double-booking of shared resources
- **Utilization Monitoring:** Inability to track usage patterns and optimize resource allocation
- **Maintenance Visibility:** No predictive maintenance for shared industrial equipment

Table 1.1: Critical Limitations of Current Industrial Remote Access Solutions

Limitation Category	Specific Problem	Business Impact
Physical Access	Engineers must be on-site to access equipment	Limited training capacity, high travel costs
VPN Complexity	Complex configuration, security vulnerabilities	IT overhead, OT security risks
Fragmented Tools	Separate systems for VMs, remote desktop, robots	Context-switching, integration difficulties
No Session Management	No automatic expiration or resource quotas	Resource conflicts, security exposure
Limited Audit Trail	Insufficient logging for compliance	IEC 62443 non-compliance
No Predictive Maintenance	Reactive maintenance only	Equipment downtime, safety risks

2 Study of the Existing

Before proposing the IoT-REAP architecture, it is essential to examine current industrial remote access solutions, understand their technical approaches, and identify the specific gaps they fail to address. This analysis establishes the foundation for the proposed platform design.

2.1 Analysis of Existing Industrial Remote Access Solutions

Industrial remote access has traditionally relied on several established approaches, each with distinct characteristics and limitations. Understanding these existing solutions is essential for positioning the IoT-REAP platform within the current technological landscape.

Virtual Private Networks (VPNs): The most common approach for industrial remote access involves establishing encrypted tunnels between remote users and industrial networks. Organizations deploy VPN concentrators at facility perimeters, requiring engineers to install client software, configure connections, and authenticate before accessing internal resources.

Jump Servers / Bastion Hosts: Some organizations implement intermediate servers that act as controlled access points to OT networks. Engineers first connect to the jump server, then establish secondary connections to target systems. This adds a security layer but increases connection complexity and latency.

Proprietary Remote Access Tools: Industrial equipment vendors (Siemens, Rockwell, Schneider) offer vendor-specific remote access solutions tied to their hardware ecosystems. These tools provide optimized connections to specific PLCs and HMIs but create vendor lock-in and fragmentation across multi-vendor environments.

Cloud-Based IIoT Platforms: Emerging platforms like AWS IoT, Azure IoT Hub, and PTC ThingWorx provide cloud connectivity for industrial devices. However, these focus primarily on data collection and monitoring rather than interactive remote control and desktop access.

Generic Remote Desktop Solutions: Tools like TeamViewer, AnyDesk, and Microsoft RDP provide remote desktop capabilities but lack industrial-specific features such as OT network isolation, session quotas, audit logging for compliance, and integration with virtualization platforms.

2.2 Criticism of the Existing

The current industrial remote access landscape exhibits several fundamental deficiencies:

VPN Complexity and Security Risks: VPN deployments require significant IT expertise for configuration and maintenance. Split tunneling creates security vulnerabilities, and VPN

credentials are prime targets for attackers. Once inside the VPN, lateral movement across the network is often unrestricted, violating zero-trust principles.

Lack of Session Management: Existing solutions provide no automated session expiration, resource quotas, or reservation systems. Shared equipment (industrial VMs, robots) cannot be efficiently scheduled, leading to conflicts and underutilization.

No OT/IT Network Isolation: Traditional remote access tools do not enforce Purdue model network segmentation. Engineers accessing OT systems from IT networks create security vulnerabilities that violate IEC 62443 standards [5].

Missing Audit Trail: Compliance with industrial security standards requires comprehensive logging of all access and commands. Existing tools lack tamper-proof audit logs with cryptographic integrity verification.

Fragmented Tooling: Organizations must deploy separate systems for VM management, remote desktop, IoT device control, and camera monitoring. This fragmentation increases operational complexity and prevents unified security policies.

No Predictive Capabilities: Existing solutions provide no demand forecasting for resource allocation or predictive maintenance for connected equipment. Administrators cannot anticipate peak usage or equipment failures.

Table 1.2: Comparison of Existing Industrial Remote Access Solutions

Requirement	VPN Solutions	Generic RDP Tools	IIoT Platforms
Browser-based Access	No, client required	Client required (limited web exceptions)	Yes, web dashboards
VM Provisioning	Not supported	Not supported	Not supported (data focus only)
Session Management	Poor: Manual only	Poor: No automation	Partial support
OT/IT Isolation	Poor: Bypasses zones	Poor: No awareness	Partial support
Audit Logging	Good: Basic logs	Poor: Insufficient	Good: Good logging
IoT Integration	Poor: Not supported	Poor: Not supported	Good: Core feature
AI Analytics	Poor: Not supported	Poor: Not supported	Good: Growing support

These deficiencies demonstrate that current solutions are designed for traditional IT environments and do not satisfy Industry 4.0 requirements for zero-trust security, OT compliance, resource-efficient scheduling, unified management, and predictive operations.

Consequently, a purpose-built industrial remote access platform is required.

3 Proposed Solution

Based on the identified deficiencies in existing solutions, this section presents the IoT-REAP platform architecture, its core functional components, security framework, and the AI/ML capabilities that enable intelligent resource management and predictive operations.

3.1 Project Definition

The IoT-REAP (Internet of Things - Remote Equipment Access Platform) project aims to develop an integrated web-based platform that provides secure remote access to industrial systems, virtual machines, and IoT devices. The project addresses the critical limitations identified in current industrial remote access solutions, creating a unified platform for Industry 4.0 operations.

Project Scope: Development of a comprehensive platform enabling engineers to provision virtual machines, access them via browser-based remote desktop, control IoT robots, monitor camera feeds, and operate within a zero-trust security framework—all from a single unified interface without client-side software installation.

Core Problem Statement: Existing remote access solutions are fragmented, lack OT-specific security controls, provide no intelligent resource management, and fail to meet industrial compliance requirements. Organizations need a purpose-built platform that unifies VM provisioning, remote desktop access, IoT control, and security monitoring.

3.2 Solution Architecture

The proposed solution employs a modern web application architecture built on established software engineering principles:

Separation of Concerns: Clear architectural boundaries between data management, business logic, and presentation layers enable independent development and maintenance of each component. The backend follows the Service-Repository pattern, ensuring controllers remain thin.

API-First Integration: External services (Proxmox VE, Apache Guacamole, MQTT broker, AI microservice) integrate through well-defined interfaces, ensuring loose coupling and flexibility to adapt to evolving infrastructure [2] [3] [4].

Event-Driven Architecture: System components communicate via Laravel Events and

WebSocket broadcasts, enabling real-time updates for session status, robot telemetry, and security alerts [6].

Microservices for AI/ML: Machine learning components (demand forecasting, anomaly detection) are deployed as separate Python FastAPI services, allowing independent scaling and technology-appropriate implementations [7].

Zero-Trust Security Model: Every access request is authenticated and authorized regardless of network location. Sessions are continuously monitored with real-time risk scoring, and high-risk activities trigger automatic termination in accordance with least-privilege principles.

4 Requirements Specification

The requirements specification translates the project problem and proposed solution into concrete actors, system services, and quality constraints. It distinguishes who interacts with the platform, what each actor must be able to do, and the non-functional qualities expected from the solution.

4.1 Identification of Actors

The platform is organized around role-based access control implemented in the application through the user roles `engineer`, `teacher`, and `admin`, in addition to the public unauthenticated state. Each actor interacts with the same platform from a different operational perspective, and each role is associated with a distinct set of services, permissions, and workflows.

Unauthenticated User (Guest) represents any visitor who has not yet authenticated in the system. This actor can browse public training paths, access legal and informational pages, consult public discussion threads in read-only mode, view training path reviews, verify public certificates, initiate authentication and registration workflows, and access public checkout result pages after external payment redirection. The guest actor therefore serves as the entry point into the platform ecosystem.

Engineer is the primary learner and remote laboratory user. This actor enrolls in training paths, consumes educational resources such as articles and videos, takes quizzes, tracks progress, writes notes, participates in discussions, posts reviews, claims certificates, and initiates payment and refund operations when a training path is monetized. Beyond the learning dimension, the engineer also provisions virtual machine sessions, opens remote desktop access through Guacamole, configures connection preferences, reserves USB devices and cameras, attaches hardware to active sessions, enters waiting queues when devices are busy, and interacts with session-bound lab resources.

Teacher is the content producer and pedagogical supervisor. This actor creates and manages training paths, modules, and training units; authors articles; uploads and manages videos and captions; creates and publishes quizzes; reviews learner interactions through analytics dashboards; moderates instructional discussions; submits training paths for administrative approval; and requests payout for generated revenue. The teacher role links educational design with operational control over learning content.

Administrator is the platform governance and infrastructure actor. This role supervises users, teacher approval, training path approval, featured training path ordering, payment-related review processes, payout validation, infrastructure supervision, Proxmox server and node management, VM assignment approval, hardware gateway verification, camera and reservation management, moderation of flagged forum content, maintenance operations, alerts, and activity logging. Administrators also have access to impersonation and compliance-sensitive actions such as suspension and compliant deletion of user accounts.

Stripe (External Payment Processor) is an external system actor that supports the platform commerce workflows. IoT-REAP integrates with Stripe through hosted checkout redirections and signed webhook callbacks to confirm payments, create internal payment records, and react to asynchronous state changes. Stripe also participates in finance workflows such as refunds and instructor payouts, where administrators trigger payout transfers and refund executions through Stripe APIs while preserving auditability and transaction integrity [8].

The interaction among these actors reveals that the platform combines four complementary dimensions: e-learning delivery, instructor authoring, virtual laboratory provisioning, and infrastructure governance. This combination distinguishes the project from a conventional learning management system because educational workflows are directly connected to remote computing and physical IoT resource access.

4.2 Functional Requirements

The platform must support the following functional requirements according to actor responsibilities and shared system behavior.

Unauthenticated User (Guest):

- Access account registration and login workflows
- Consult the landing page and public legal pages
- Browse public training paths and training path details
- Read public reviews and forum threads in read-only mode
- Verify and download publicly accessible certificates through verification links

- View public search results, trending queries, and category-based discovery
- Access checkout success and cancellation pages after external payment flows

Engineer:

- Enroll in and unenroll from training paths
- Access owned or enrolled training units and learning materials
- Read articles and track article completion
- Stream instructional videos and update playback progress
- Attempt quizzes, submit answers, and review attempt history
- Mark training units complete or incomplete based on learning progress
- Create, update, and delete personal notes for training units
- Create, edit, and delete reviews for completed or relevant training paths
- Participate in forum discussions by creating threads, replying, voting, and flagging content
- View notifications and manage read status
- Claim, verify, and download certificates
- Initiate checkout for paid training paths
- Review payment history and submit refund requests
- Provision, extend, view, and terminate virtual machine sessions
- Retrieve Guacamole access tokens for remote desktop connectivity
- Manage connection preference profiles for available protocols
- Browse available Proxmox virtual machines and snapshots
- Reserve USB devices and cameras
- Attach or detach session hardware and join hardware waiting queues
- Access session camera streams, request control, move PTZ-enabled cameras, and change stream resolution

Teacher:

- Create, edit, archive, restore, and delete training paths
- Organize training paths into modules and training units
- Reorder modules and training units
- Create and maintain quiz structures, questions, and publication status
- Create, update, and delete article content
- Upload, retry, and delete training videos
- Manage video captions
- Submit training paths for administrative review
- View teacher analytics for enrollment, earnings, and learning funnel data
- Access a teacher discussion inbox and moderate course-related threads
- Request revenue payout
- Submit training unit virtual machine assignment requests

Administrator:

- Access the administrative dashboard and KPI views
- Approve, reject, feature, unfeature, and order training paths
- Review pending VM assignment requests
- Manage reservations for devices and cameras
- Manage Proxmox servers and nodes
- Start, stop, reboot, and shut down virtual machines through node operations
- Register, verify, update, and remove gateway nodes
- Discover hardware devices and refresh infrastructure status
- Bind, unbind, attach, detach, dedicate, and configure devices and cameras
- Assign and unassign cameras to virtual machines
- Approve teacher accounts and manage user roles
- Suspend, unsuspend, impersonate, and delete user accounts where authorized

- Review and process refund requests
- Review, approve, reject, and process payout requests
- Monitor video processing status
- Place devices and cameras in maintenance mode
- Moderate flagged threads and replies
- Manage system alerts and consult activity logs

System-Wide Functional Requirements:

- Enforce authentication and email verification for protected features
- Apply authorization policies according to role and ability
- Persist learning, payment, infrastructure, and moderation data
- Support public and protected routes through a unified web interface
- Provide rate limiting for sensitive or abuse-prone operations
- Maintain notification and audit mechanisms for critical events
- Support asynchronous or stateful workflows such as video processing, VM lifecycle operations, refund handling, and hardware attachment

4.3 Non-Functional Requirements

The quality constraints of the platform are defined by the educational, infrastructure, and operational nature of the system.

Performance:

- The platform must provide responsive navigation across training, administration, and infrastructure pages with page load times compatible with standard web application thresholds
- Video streaming and progress tracking must occur with latency compatible with interactive remote desktop and streaming workloads
- VM provisioning and session retrieval must remain usable under concurrent demand up to the tested load targets

- Search, analytics summaries, and notification retrieval must execute within bounded query response times

Security:

- Authentication must rely on secure session-based identity management and verified email addresses
- Authorization must be enforced by middleware and policy gates
- Payment workflows must validate external webhook signatures and preserve transaction integrity
- Access to VM sessions, cameras, and USB devices must be restricted to authorized users through role-based policies
- Sensitive administrative actions must be auditable with immutable log entries

Reliability:

- Learning progress, notes, payments, and reservations must remain durable and consistent across system restarts
- Hardware and infrastructure actions must tolerate transient failure and preserve observable state
- Session resources must support lifecycle management including creation, extension, and cleanup
- Alerting and logging must assist recovery from operational issues

Usability:

- The user interface must remain coherent for learners, teachers, and administrators
- Learning workflows must be understandable without specialized infrastructure knowledge
- Administrative screens must expose complex operations in a controlled and structured form
- Forum, notes, reviews, and certificate features must integrate naturally into the learning experience

Scalability and Maintainability:

- The architecture must support extension through modular routes, controllers, models, and services
- Domain separation between learning, commerce, virtual lab, and operations must remain clear
- The platform must support future growth in training content, users, devices, and compute nodes
- The codebase must remain testable and maintainable through strongly typed frontend code and structured backend services

5 Work Methodology

The development of IoT-REAP required a structured approach to coordinate complex technical requirements with evolving stakeholder needs. This section presents the Agile Scrum methodology framework and the specific development timeline employed throughout the project.

5.1 The Agile Scrum Methodology

This subsection explains why Scrum was selected for IoT-REAP and how its roles, ceremonies, and sprint structure organize the development work before the detailed definition below.

5.1.1 Definition

Agile Scrum is a concrete, structured framework for implementing Agile principles in software development. While Agile is a broader philosophy emphasizing iterative delivery and flexibility, Scrum is a specific instantiation of Agile that defines roles, ceremonies, and artifacts. Scrum breaks projects into fixed-duration iterations called **sprints** (typically 1-2 weeks), during which a cross-functional team commits to delivering a potentially shippable product increment. Each sprint cycles through planning, daily standups, development, and review ceremonies, with retrospectives enabling continuous process improvement.

Scrum defines three key roles: **Product Owner** (prioritizes backlog and validates requirements), **Scrum Master** (facilitates ceremonies and removes blockers), and **Development Team** (delivers the increment). The framework manages work through three artifacts: **Product Backlog** (prioritized features and requirements), **Sprint Backlog** (selected items for current sprint), and **Increment** (working software produced at sprint end).

The Agile Scrum framework emerges from the Agile Manifesto (2001), which prioritizes:

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

For the IoT-REAP platform, Agile Scrum is particularly appropriate given the need to balance evolving requirements, stakeholder feedback, and technical innovation in AI integration. The structured ceremonies and defined roles provide accountability, while the sprint structure enables regular milestone verification and MVP validation.

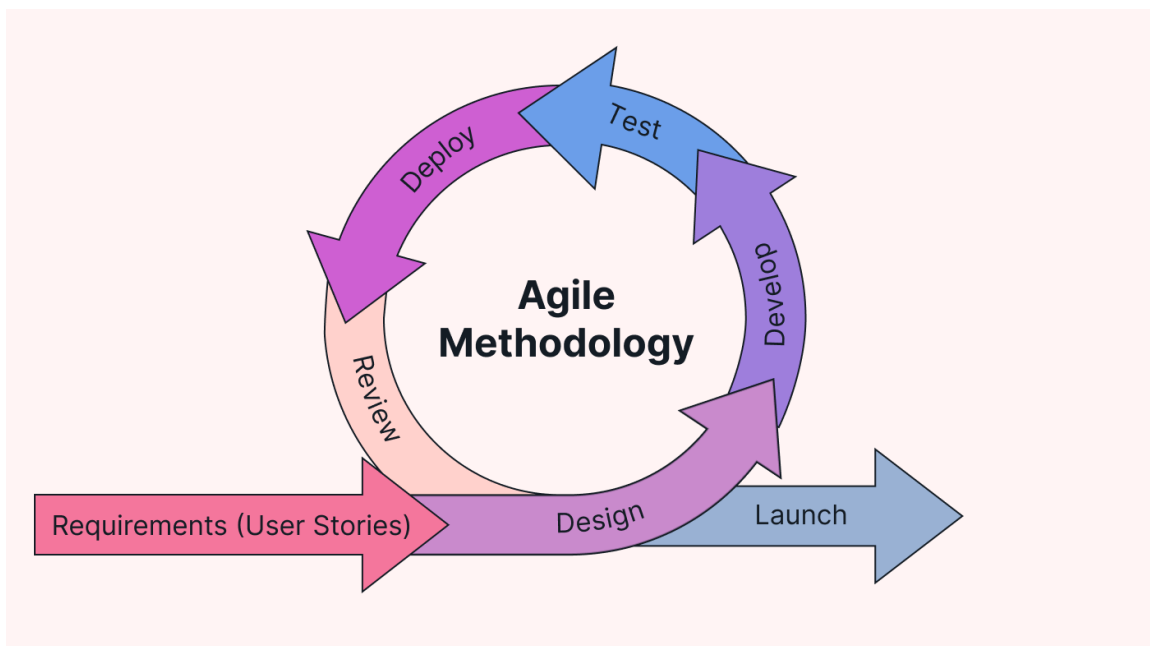


Figure 1.2: Agile Scrum Methodology Workflow

5.1.2 Characteristics of Agile Scrum

Sprint-Based Delivery: The IoT-REAP project is organized into fixed 2-week sprints, each with a clear starting point (Sprint Planning), daily synchronization (Daily Standup), and endpoint (Sprint Review & Retrospective). This structure provides predictable delivery cycles and regular stakeholder engagement.

Sprint Planning & Commitment: At the beginning of each sprint, the team (in the solo developer context, this involves supervisors and stakeholders) commits to specific user stories and tasks. Sprint planning defines clear acceptance criteria, ensuring understanding of requirements before development begins.

Product Backlog Management: Features and requirements are maintained in a prioritized Product Backlog of user stories. The Product Owner role prioritizes items based on business

value, risk, and dependencies. This allows adaptation: new security requirements or infrastructure changes are incorporated into subsequent sprints.

Sprint Increments: Each sprint produces a working increment of the software. Early sprints deliver authentication and VM provisioning; middle sprints add remote desktop and robot control; later sprints implement AI scheduling and compliance features. This incremental approach enables early validation through the Sprint 3 MVP demo.

Daily Standups & Transparency: Regular synchronization meetings (adapted to weekly supervisor meetings in this solo context) maintain transparency on progress, blockers, and dependencies. This enables rapid problem-solving and prevents misalignment.

Sprint Review & Stakeholder Feedback: At each sprint boundary, completed work is demonstrated to stakeholders. This enables immediate validation, ensures features meet needs, and allows requirement refinement for subsequent sprints.

Sprint Retrospective & Process Improvement: After each sprint review, the team conducts retrospectives to identify what goes well, what can improve, and what to change. This continuous improvement mindset enhances developer productivity, code quality, and stakeholder satisfaction over successive iterations.

Definition of Done: Scrum requires clear acceptance criteria for task completion. For IoT-REAP, "done" means: code written, unit tests passing (target 80% coverage), code reviewed against project guidelines, feature documented, and acceptance criteria validated.

5.2 Product Backlog

The IoT-REAP platform development followed Agile Scrum principles with a prioritized product backlog organized by sprint. The complete backlog comprises user stories. Table 1.3 presents key user stories with priorities and sprint assignments. Sprints 2.5 and 4.5 represent mid-cycle scope extensions identified during backlog refinement, executed without disrupting the main two-week cadence.

Table 1.3: IoT-REAP Platform Product Backlog

ID	User Story / Feature	Priority	Sprint
Sprint 1: Foundation & Authentication			
01	As a user, I want to register and log in securely using session-based authentication	High	Sprint 1
02	As an admin, I want role-based access control (engineer, admin, security_officer)	High	Sprint 1
03	As a developer, I need database schema for users with ULID primary keys	High	Sprint 1

Table 1.3

ID	User Story / Feature	Priority	Sprint
04	As a user, I want password reset functionality via email	Medium	Sprint 1
05	As a developer, I need React frontend with TypeScript and Zustand state management [9] [10] [11]	High	Sprint 1
Sprint 2: Proxmox Integration			
06	As a developer, I need database schema for VM nodes, templates, and sessions	High	Sprint 2
07	As a developer, I need Proxmox API client with retry logic and error handling	High	Sprint 2
08	As an engineer, I want to browse VM templates and launch sessions	High	Sprint 2
09	As a developer, I need load balancer to distribute VMs across Proxmox nodes	High	Sprint 2
10	As an admin, I want to view all nodes with live CPU/RAM stats	Medium	Sprint 2
11	As an admin, I want to register a Proxmox server with encrypted credentials	High	Sprint 2

Table 1.3

ID	User Story / Feature	Priority	Sprint
Sprint 2.5: Multi-Proxmox Server Support			
11a	As a developer, I need to extend the Proxmox architecture to support multiple independent clusters	High	Sprint 2.5
11b	As an engineer, I want to select a target Proxmox cluster when provisioning a VM	High	Sprint 2.5
Sprint 3: Guacamole Remote Access — MVP			
12	As a developer, I need Guacamole client service for connection management	High	Sprint 3
13	As an engineer, I want browser-based RDP/VNC access to my VM session	High	Sprint 3
14	As an engineer, I want to extend my session duration (+30 min increments)	High	Sprint 3
15	As an engineer, I want to terminate my session and have the VM cleaned up	High	Sprint 3
16	As an engineer, I want to save my preferred connection settings (display, audio)	Medium	Sprint 3
Sprint 4: File Transfer & Robot Foundation			
17	As a developer, I need MQTT service for robot command publishing	High	Sprint 4
18	As an engineer, I want to reserve a robot with atomic transaction locks	High	Sprint 4
19	As an engineer, I want to control the robot using Blockly visual programming	High	Sprint 4
20	As an engineer, I want an emergency stop button that works instantly	High	Sprint 4
21	As an engineer, I want real-time robot telemetry via WebSocket	High	Sprint 4
Sprint 4.5: Technical Training Platform			
41	As an instructor, I want to create structured courses with modules, lessons, and prerequisite tracking	High	Sprint 4.5
42	As an instructor, I want to upload videos with HLS streaming and multi-quality playback (360p-1080p) [12]	High	Sprint 4.5
43	As a learner, I want to enroll in courses and track my progress across lessons and modules	High	Sprint 4.5
44	As an instructor, I want to create timed quizzes with auto-grading, passing scores, and retry limits	High	Sprint 4.5

Table 1.3

ID	User Story / Feature	Priority	Sprint
45	As a learner, I want hands-on VM lab sessions integrated with course lessons for practical training	High	Sprint 4.5
46	As a learner, I want to receive verifiable digital certificates upon course completion	High	Sprint 4.5
47	As a learner, I want to take timestamped notes during video lessons for later review	Medium	Sprint 4.5
48	As a learner, I want to rate and review courses to help other learners	Medium	Sprint 4.5
49	As an instructor, I want analytics dashboard showing enrollments, completions, and revenue	High	Sprint 4.5
50	As an admin, I want to manage course categories and featured course placement	Medium	Sprint 4.5
Sprint 5: Camera & AI Scheduler			
22	As a developer, I need Python FastAPI microservice for ML demand forecasting	High	Sprint 5
23	As an admin, I want 24-hour VM demand predictions with auto-refresh	High	Sprint 5
24	As a developer, I need idle session hibernation (suspend after 10 min inactivity)	Medium	Sprint 5
25	As an engineer, I want live camera feed from ESP32-CAM during robot control	High	Sprint 5
Sprint 6: Zero-Trust Security — Feature Complete			
26	As a developer, I need device fingerprinting service	High	Sprint 6
27	As a developer, I need geo-IP anomaly detection using MaxMind	High	Sprint 6
28	As a developer, I need predictive maintenance AI for robot health scoring	High	Sprint 6

Table 1.3

ID	User Story / Feature	Priority	Sprint
Sprint 7: OT Isolation, Catalog & Compliance			
29	As an admin, I want to enforce Purdue model network isolation (IT/OT zones)	High	Sprint 7
30	As a developer, I need tamper-proof audit log with SHA-256 hash chain	High	Sprint 7
31	As an admin, I want PDF compliance reports for IEC 62443 audits	High	Sprint 7
32	As an engineer, I want AI-assisted template suggestions based on my description	Medium	Sprint 7
33	As an admin, I want complete dashboard with quota management and activity logs	High	Sprint 7
Sprint 8: Testing			
34	As a developer, I need 80% test coverage with mocked external APIs	High	Sprint 8
35	As a developer, I need load testing: 50 concurrent logins, 25 VMs, 40 Guacamole sessions	High	Sprint 8
36	As a user, I want comprehensive documentation (user manual, API docs)	High	Sprint 8
37	As a developer, I need to prepare and deliver 15-minute live demo	High	Sprint 8

5.3 Agile Scrum Ceremonies and Collaboration

Sprint Planning (Duration: 2-4 hours): At the beginning of each 2-week sprint, sprint planning meetings engage the developer, supervisors, and Advisory Unit stakeholders. Sessions define sprint goals, select user stories from the prioritized Product Backlog (based on business value and dependencies), and break them into implementation tasks. Each user story receives effort estimation using story points, resulting in a Sprint Backlog of committed work with clear acceptance criteria. This ceremony ensures shared understanding of requirements and realistic commitment.

Daily Standup (Duration: 15 minutes): Although conducted as a solo developer, daily progress tracking maintains transparency. Weekly formal standups (Mondays) with supervisors answer three questions: What was completed? What will be done? What obstacles exist? This adaptation of the daily ceremony pattern provides checkpoint accountability while respecting the solo context.

Sprint Review & Demo (Duration: 1-2 hours): At each sprint's end, completed work is demonstrated to stakeholders. Reviewers examine running features, validate acceptance criteria

compliance, and provide feedback influencing subsequent sprint prioritization. The Sprint 3 review includes a formal MVP (Minimum Viable Product) demo showcasing VM provisioning, remote desktop access, and basic robot control—a critical milestone confirming core platform viability.

Sprint Retrospective (Duration: 1 hour): Following the review, the team conducts retrospectives examining what works well, what can improve, and specific action items for the next sprint. Early retrospectives identify: need for more comprehensive API documentation, benefit of Copilot-assisted development, and infrastructure bottlenecks requiring architectural adjustments. These insights are systematically incorporated into subsequent sprints, improving productivity and code quality.

Product Backlog Refinement (Ongoing): Between sprints, the Product Backlog is continuously refined. New requirements (additional security measures, infrastructure changes, performance optimizations) are added, prioritized, and estimated. This preparation ensures Sprint Planning sessions are efficient and focused on high-value items.

5.4 The UML Modeling Language



Figure 1.3: UML modeling language

Unified Modeling Language (UML) is a standardized visual modeling language used in software engineering to specify, visualize, construct, and document the artifacts of software systems [14]. Developed by Grady Booch, James Rumbaugh, and Ivar Jacobson in the mid-1990s, UML provides a common vocabulary for object-oriented software design and is the industry standard for software architecture documentation.

For the IoT-REAP platform, UML modeling was applied during the design phase to produce use case diagrams (Section 4), class diagrams, and sequence diagrams illustrating critical workflows such as VM provisioning and robot command dispatch. These models served as architectural contracts between the developer and stakeholders, ensuring shared understanding of system boundaries before implementation.

UML consists of multiple diagram types that capture different aspects of system design:

- **Structural Diagrams:** Class diagrams, component diagrams, deployment diagrams, package diagrams
- **Behavioral Diagrams:** Use case diagrams, sequence diagrams, activity diagrams, state machine diagrams

6 Project Planning

The IoT-REAP platform development is organized as a phased timeline from February to mid-June 2026, following an Agile Scrum sprint-based approach [15]. The project is structured into eight two-week sprints with a small integration and testing buffer, and it reaches two key milestones at Sprint 3 (MVP Demo) and Sprint 6 (Feature Complete).

6.1 Sprint-Based Development Phases

The following phases present the project timeline in chronological order, showing how the platform evolved from foundation and integration work to testing and final delivery.

Sprint 1: Foundation & Authentication (February 2-15, 2026)

Establishes project foundation with Laravel 12 and React 18 integration via Breeze. Implements session-based authentication, role-based access control (engineer, admin, security_officer), and database schema with ULID primary keys. Sets up MySQL, Redis, and Docker development environment [16] [17] [18].

Sprint 2: Proxmox Integration (February 16 - March 1, 2026)

Builds Proxmox VE API client wrapper with retry logic and error handling. Creates database schema for VM nodes, templates, and sessions. Implements load balancer for VM distribution across nodes and admin node management dashboard with live stats.

Sprint 2.5: Multi-Proxmox Server Support (March 2-8, 2026)

Extends architecture to support multiple independent Proxmox clusters. Implements encrypted credential storage, server registration API, and cluster selection logic for engineers launching VMs.

Sprint 3: Guacamole Remote Access — MVP Demo (March 9-22, 2026)

Deploys Apache Guacamole and integrates its API with Laravel. Implements connection creation, one-time token generation, and embedded viewer in React. Adds session extension,

termination, and connection preference persistence. **This sprint culminates in the MVP demonstration.**

Sprint 4: File Transfer & Robot Foundation (March 23 - April 5, 2026)

Builds MQTT infrastructure for robot command publishing with QoS 1. Creates atomic robot reservation system preventing double-booking. Deploys Blockly visual programming interface and implements real-time telemetry streaming via WebSocket.

Sprint 4.5: Technical Training Platform (April 6-19, 2026)

Develops comprehensive learning management system for industrial skills training. Implements course structure with modules, lessons (video, reading, practice, VM-lab types), and prerequisite tracking. Builds HLS video streaming with adaptive quality (360p-1080p) and progress tracking. Creates timed quiz system with auto-grading, passing scores, and retry limits. Integrates VM lab sessions directly into course lessons for hands-on practice. Implements digital certificate generation with unique verification hashes.

Sprint 5: Camera & AI Scheduler (April 20 - May 3, 2026)

Deploys Python FastAPI microservice for ML-based demand forecasting using RandomForestRegressor. Integrates predictions with Laravel scheduler for VM pre-warming. Adds live ESP32-CAM video feed and idle session hibernation [7] [6].

Sprint 6: Zero-Trust Security — Feature Complete (May 4-17, 2026)

Implements device fingerprinting and geo-IP anomaly detection. Builds real-time risk scoring engine with automatic session termination for high-risk activities. Trains IsolationForest model for robot health anomaly detection. **All core backlog items are implemented at this checkpoint.**

Sprint 7: OT Isolation, Catalog & Compliance (May 18-31, 2026)

Enforces Purdue model network isolation with IT/OT zone separation. Builds tamper-proof audit log with SHA-256 hash chain for IEC 62443 compliance. Implements PDF compliance report generator and AI-assisted template catalog [5].

Sprint 8: Testing (June 1-14, 2026)

Achieves 80% test coverage with mocked external APIs. Conducts load testing (50 concurrent logins, 25 VMs, 40 Guacamole sessions). Executes OWASP ZAP security scan. Prepares documentation and rehearses defense presentation.

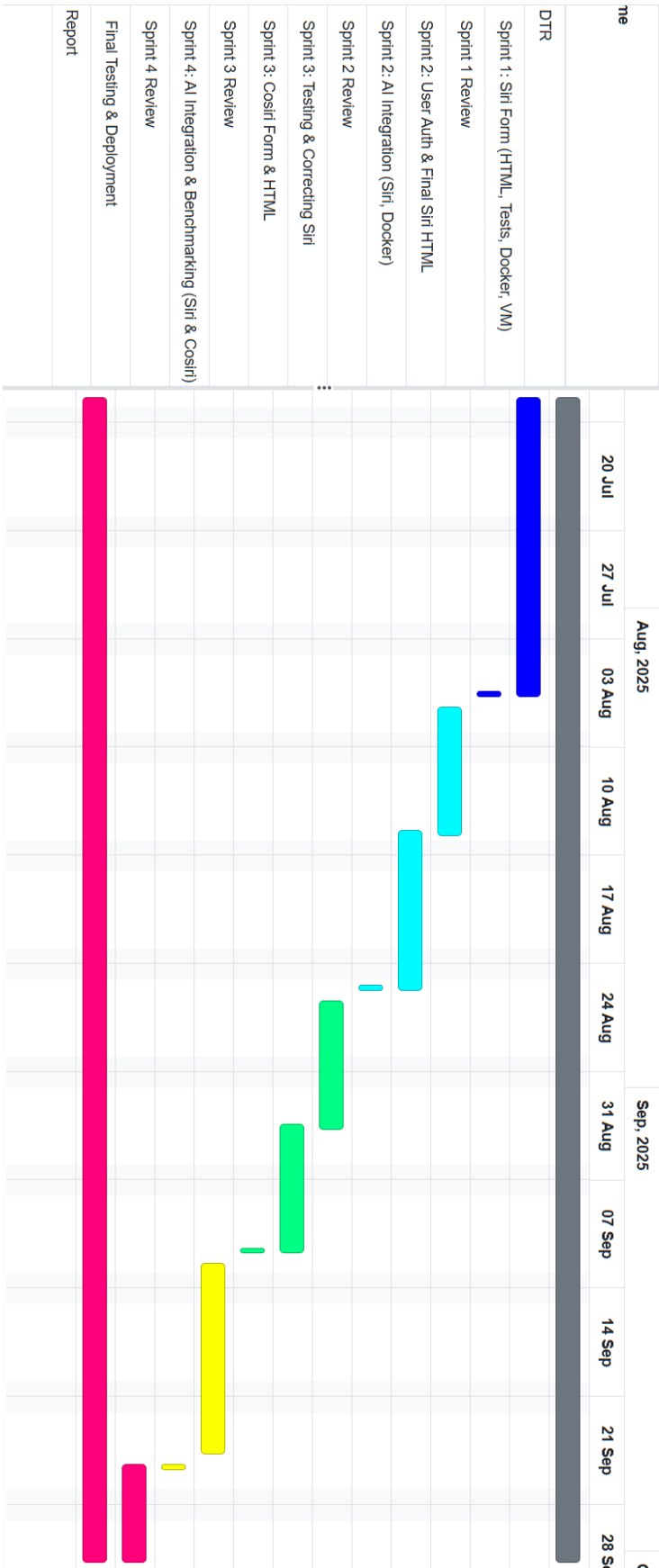


Figure 1.4: IoT-REAP Platform Development Timeline and Milestones

Conclusion

This chapter has presented the project context, analyzed current remote access limitations, introduced the IoT-REAP solution, specified the functional and non-functional requirements, and described the Agile Scrum methodology and sprint planning adopted to structure the implementation.

Webgraphy

- [1] Novation City, “Novation City,” *Novation City*, <https://novationcity.com>, Accessed: April 23, 2026.
- [2] Proxmox Server Solutions GmbH, “Proxmox VE API,” *Proxmox VE Wiki*, https://pve.proxmox.com/wiki/Proxmox_VE_API, Accessed: February 15, 2026.
- [3] The Apache Software Foundation, “Apache Guacamole Manual,” *Apache Guacamole*, <https://guacamole.apache.org/doc/gug/>, Accessed: February 4, 2026.
- [4] OASIS Open, “MQTT Version 5.0,” *OASIS Open*, <https://www.oasis-open.org/standard/mqtt-v5-0/>, Accessed: May 6, 2026.
- [5] International Electrotechnical Commission, “Understanding IEC 62443,” *IEC*, <https://www.iec.ch/blog/understanding-iec-62443>, Accessed: March 8, 2026.
- [6] Laravel LLC, “Laravel 12.x Documentation,” *Laravel*, <https://laravel.com/docs/12.x>, Accessed: March 4, 2026.
- [7] Sebastian Ramirez, “FastAPI Documentation,” *FastAPI*, <https://fastapi.tiangolo.com/>, Accessed: March 1, 2026.
- [8] Stripe, Inc., “Stripe Documentation,” *Stripe Docs*, <https://docs.stripe.com/>, Accessed: February 18, 2026.
- [9] Meta Open Source, “React Documentation,” *React*, <https://react.dev/>, Accessed: May 6, 2026.
- [10] Microsoft Corporation, “TypeScript Documentation,” *TypeScript*, <https://www.typescriptlang.org/docs/>, Accessed: February 14, 2026.
- [11] pmndrs, “Zustand Documentation,” *Zustand*, <https://zustand.docs.pmnd.rs/>, Accessed: April 28, 2026.
- [12] RFC Editor, “RFC 8216: HTTP Live Streaming,” *RFC Editor*, <https://www.rfc-editor.org/rfc/rfc8216>, Accessed: May 6, 2026.
- [13] OWASP Foundation, “Zed Attack Proxy Documentation,” *OWASP ZAP*, <https://www.zaproxy.org/docs/>, Accessed: May 26, 2026.
- [14] Grady Booch, James Rumbaugh, and Ivar Jacobson, “The Unified Modeling Language User Guide, 2nd ed.,” *Addison-Wesley*, https://personal.utdallas.edu/~chung/Fujitsu/UML_2.0/Rumbaugh--UML_2.0_Reference_CD.pdf, Accessed: April 11, 2026.

- [15] Scott W. Ambler, “Agile Documentation,” *Agile Modeling*, <https://agilemodeling.com/essays/agiledocumentation.htm>, Accessed: February 12, 2026.
- [16] Oracle Corporation, “MySQL 8.4 Reference Manual,” *MySQL Documentation*, <https://dev.mysql.com/doc/refman/8.4/en/>, Accessed: April 17, 2026.
- [17] Redis Ltd., “Redis Documentation,” *Redis Docs*, <https://redis.io/docs/latest/>, Accessed: March 27, 2026.
- [18] Docker, Inc., “Docker Documentation,” *Docker Docs*, <https://docs.docker.com/>, Accessed: February 5, 2026.
- [19] Martin Fowler, “Patterns of Enterprise Application Architecture,” *Enterprise Application Architecture*, <https://dl.ebooksworld.ir/motoman/Patterns%20of%20Enterprise%20Application%20Architecture.pdf>, Accessed: February 4, 2026.
- [20] Microsoft Corporation, “Visual Studio Code Documentation,” *Visual Studio Code*, <https://code.visualstudio.com/docs>, Accessed: February 12, 2026.
- [21] Laravel LLC, “Laravel Herd Documentation,” *Laravel Herd*, <https://herd.laravel.com/docs/windows/getting-started/installation>, Accessed: February 28, 2026.
- [22] Software Freedom Conservancy, “Git Documentation,” *Git*, <https://git-scm.com/docs>, Accessed: March 2, 2026.
- [23] GitHub, Inc., “GitHub Docs,” *GitHub Docs*, <https://docs.github.com/en>, Accessed: April 6, 2026.
- [24] Nils Adermann and Jordi Boggiano, “Composer Documentation,” *Composer*, <https://getcomposer.org/doc/>, Accessed: April 19, 2026.
- [25] npm, Inc., “npm Docs,” *npm Docs*, <https://docs.npmjs.com/>, Accessed: February 4, 2026.
- [26] Postman, Inc., “Postman Docs,” *Postman Learning Center*, <https://learning.postman.com/>, Accessed: April 13, 2026.
- [27] JGraph Ltd., “diagrams.net,” *diagrams.net*, <https://www.diagrams.net/>, Accessed: February 26, 2026.
- [28] OWASP Foundation, “Zed Attack Proxy Documentation,” *OWASP ZAP*, <https://www.zaproxy.org/docs/>, Accessed: June 3, 2026.

- [29] Cisco Systems, Inc., “Cisco Packet Tracer Documentation,” *Cisco Networking Academy*, <https://www.netacad.com/cisco-packet-tracer>, Accessed: May 12, 2026.
- [30] Broadcom Inc., “VMware Technical Documentation,” *Broadcom TechDocs*, <https://techdocs.broadcom.com/>, Accessed: April 3, 2026.
- [31] The PHP Group, “PHP Manual,” *PHP Documentation*, <https://www.php.net/docs.php>, Accessed: May 3, 2026.
- [32] Inertia.js, “Inertia.js Documentation,” *Inertia.js*, <https://inertiajs.com/>, Accessed: April 25, 2026.
- [33] Laravel LLC, “Laravel Fortify Documentation,” *Laravel*, <https://laravel.com/docs/12.x/fortify>, Accessed: May 1, 2026.
- [34] Laravel LLC, “Laravel Socialite Documentation,” *Laravel*, <https://laravel.com/docs/12.x/socialite>, Accessed: April 11, 2026.
- [35] Google, “Using OAuth 2.0 to Access Google APIs,” *Google Identity*, <https://developers.google.com/identity/protocols/oauth2>, Accessed: March 26, 2026.
- [36] Laravel LLC, “Laravel Reverb Documentation,” *Laravel*, <https://laravel.com/docs/12.x/reverb>, Accessed: March 1, 2026.
- [37] Laravel LLC, “Laravel Telescope Documentation,” *Laravel*, <https://laravel.com/docs/12.x/telescope>, Accessed: March 30, 2026.
- [38] Vite, “Vite Documentation,” *Vite*, <https://vite.dev/guide/>, Accessed: April 17, 2026.
- [39] Tailwind Labs, “Tailwind CSS Documentation,” *Tailwind CSS*, <https://tailwindcss.com/docs>, Accessed: March 8, 2026.
- [40] WorkOS, “Radix UI Documentation,” *Radix UI*, <https://www.radix-ui.com/primitives/docs/overview/introduction>, Accessed: May 15, 2026.
- [41] React Hook Form, “React Hook Form Documentation,” *React Hook Form*, <https://react-hook-form.com/docs>, Accessed: May 23, 2026.
- [42] TanStack, “TanStack Query Documentation,” *TanStack*, <https://tanstack.com/query/latest/docs/framework/react/overview>, Accessed: February 1, 2026.

- [43] Recharts Group, “Recharts Documentation,” *Recharts*, <https://recharts.org/en-US/>, Accessed: May 9, 2026.
- [44] bluenviron, “MediaMTX,” *GitHub*, <https://github.com/bluenviron/mediamtx>, Accessed: May 15, 2026.
- [45] RFC Editor, “RFC 2326: Real Time Streaming Protocol,” *RFC Editor*, <https://www.rfc-editor.org/rfc/rfc2326>, Accessed: February 21, 2026.
- [46] World Wide Web Consortium, “WebRTC: Real-Time Communication in Browsers,” *W3C Recommendation*, <https://www.w3.org/TR/webrtc/>, Accessed: May 1, 2026.
- [47] Frigate NVR, “Frigate Documentation,” *Frigate*, <https://docs.frigate.video/>, Accessed: March 27, 2026.
- [48] FFmpeg, “FFmpeg Documentation,” *FFmpeg*, <https://ffmpeg.org/documentation.html>, Accessed: March 16, 2026.
- [49] IETF, “WebRTC-HTTP Egress Protocol (WHEP),” *IETF Datatracker*, <https://datatracker.ietf.org/doc/draft-ietf-wish-whep/>, Accessed: March 8, 2026.
- [50] Eclipse Foundation, “Eclipse Mosquitto Documentation,” *Eclipse Mosquitto*, <https://mosquitto.org/documentation/>, Accessed: February 20, 2026.
- [51] The Linux Kernel Organization, “USB/IP Protocol,” *Linux Kernel Documentation*, https://docs.kernel.org/usb/usbip_protocol.html, Accessed: February 28, 2026.
- [52] Sebastian Bergmann, “PHPUnit Documentation,” *PHPUnit*, <https://docs.phpunit.de/>, Accessed: May 9, 2026.
- [53] Vitest, “Vitest Documentation,” *Vitest*, <https://vitest.dev/guide/>, Accessed: March 16, 2026.
- [54] OVHcloud, “VPS,” *OVHcloud*, <https://www.ovhcloud.com/en/vps/>, Accessed: February 14, 2026.
- [55] Internet Security Research Group, “Let’s Encrypt Documentation,” *Let’s Encrypt*, <https://letsencrypt.org/docs/>, Accessed: February 12, 2026.
- [56] Canonical Ltd., “Ubuntu Documentation,” *Ubuntu Help*, <https://help.ubuntu.com/>, Accessed: March 21, 2026.
- [57] F5, Inc., “nginx Documentation,” *nginx*, <https://nginx.org/en/docs/>, Accessed: February 13, 2026.

- [58] Electronic Frontier Foundation, “Certbot Documentation,” *Certbot*, <https://eff-certbot.readthedocs.io/en/stable/>, Accessed: March 18, 2026.
- [59] Mozilla, “Strict-Transport-Security,” *MDN Web Docs*, <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Strict-Transport-Security>, Accessed: May 20, 2026.
- [60] Mozilla, “Content Security Policy (CSP),” *MDN Web Docs*, <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP>, Accessed: March 17, 2026.

Abstract

This report presents IoT-REAP, a secure industrial remote operations platform for Industry 4.0. The platform addresses challenges in remote access, such as VPN complexity, session management, and OT/IT network isolation.

Developed with Laravel, React, TypeScript, and Python FastAPI, IoT-REAP integrates Proxmox VE for VM provisioning, Apache Guacamole for browser-based remote desktop, and MQTT for IoT robot control. Innovations include zero-trust security, device fingerprinting, geo-IP anomaly detection, real-time risk scoring, AI-powered demand forecasting, and predictive maintenance.

The system supports 50+ concurrent users with automatic VM provisioning, sub-200ms robot telemetry latency, and tamper-proof audit logging for IEC 62443 compliance. Results show secure, browser-based access to industrial systems with no client installation.

Keywords: Industry 4.0, IoT, Remote Access, Proxmox VE, Apache Guacamole, MQTT, Zero-Trust Security, IEC 62443, Predictive Maintenance, Machine Learning

Résumé

Ce rapport présente IoT-REAP, une plateforme sécurisée d'opérations industrielles à distance pour l'Industrie 4.0. Elle répond aux défis de l'accès distant, comme la complexité des VPN, la gestion des sessions et l'isolation des réseaux OT/IT.

Développée avec Laravel, React, TypeScript et Python FastAPI, IoT-REAP intègre Proxmox VE pour le provisionnement de VM, Apache Guacamole pour l'accès bureau à distance, et MQTT pour le contrôle de robots IoT. Les innovations incluent la sécurité zero-trust, empreinte d'appareil, détection d'anomalies géo-IP, scoring de risque, prévision de demande par IA et maintenance prédictive.

Le système supporte plus de 50 utilisateurs simultanés avec provisionnement automatique de VM, latence de télémétrie robot <200ms, et journalisation d'audit inviolable pour la conformité IEC 62443. Les résultats montrent un accès sécurisé, via navigateur, aux systèmes industriels sans installation client.

Mots-clés: Industrie 4.0, IoT, Accès à Distance, Proxmox VE, Apache Guacamole, MQTT, Sécurité Zero-Trust, IEC 62443, Maintenance Prédictive, Apprentissage Automatique